



Solo Presentation Training

Abdul Rafay — full practice guide, slide by slide, with Urdu help words.

Presenter

Abdul Rafay

Deck

15 slides · ~7 min

Event

**Alibaba AI
Hackathon 2026**

How to use this book

This book trains you to present the **VisionGuard ULTRA solo deck** (the 15-slide PDF). Simple English is used; hard words have Urdu meaning in brackets.

1. **Part 2** — what to say on every slide, with time.
2. **Part 3** — Q&A drill: practise these 12 questions until answers are short.
3. **Part 4** — rehearsal plan and score sheet.
4. **Part 5** — the full knowledge guide (every part of the project, from start to end).

GOLDEN RULE

Your **story** wins, not perfect English. Speak slowly, short sentences, look at the judges.

NUMBERS RULE

Only say numbers that are in the deck. Weapon and violence accuracy and false-alarm rate are **not final** — never invent (گھڑنا) them.

Slide-by-slide script

01

0:20

Title

"Assalam-o-Alaikum. My name is Abdul Rafay, team lead of VisionGuard. I built the main AI system. My team is Moiz, who trained our models, Areeba, who prepared and tested the data, and Aqib, who made the website."

Smile. Say your name slowly. Then pause 1 second before the next slide.

02

0:40

Problem

"Karachi has more than 20 million people and thousands of cameras. But cameras only record — nobody is really watching. An operator gets tired after about 20 minutes. So the camera sees the crime, but nobody acts."

Stop for 2 seconds after "nobody acts". Look at the judges, not the screen.

03

0:30

Solution

"VisionGuard makes cameras smart. It watches all cameras together, finds the danger, gives a risk score, saves proof, and sends the alert to the right department. Detect, decide, dispatch."

Point to the three boxes one by one while saying the three words.

04

0:20

12 detectors

"We cover twelve kinds of emergency — from fire and fights to accidents, crowds and bags left alone."

Do not read all twelve. Say 3–4, then move on.

05

2:00

Live demo

"Let me show you live. This is the control room... Here a fire is found — it goes to Fire Brigade 16... Here a fight — see the body points... Every event gets a risk score... and evidence is sealed with a fingerprint."

Switch to the app. Tell judges where to look. If slow: "While it loads, let me explain..."

06

1:00

How it works

"One frame goes in: YOLO finds objects, ByteTrack gives each person an ID, special models check pose, fire and weapons, and the event engine checks rules over time — not one frame."

Use your hand to follow the arrows left to right.

07

0:40

Models

"Moiz used transfer learning — we took YOLOv8, which already knows general shapes, and trained it more on fire, weapon and violence data on Colab GPUs."

Give Moiz credit by name.

08

0:30

Data quality

"Areeba scanned sixteen thousand fire images and seven thousand weapon images, removed duplicates, made four thousand weather images, and tuned three settings from real false alarms."

Say "data-driven" — we changed settings because of evidence.

09

0:40

Results

"Our fire model is two times better than the old colour method — precision 47.6 percent. The dataset is hard, with tiny fires, and our time checks catch fires across many frames. Weapon and violence reports are in progress."

Be honest and calm. Never round numbers up.

10

0:30

Performance

"Our async design made it 43 percent faster on a small 4 GB GPU, and dropped half the lagging frames."

Say "measured on our test GPU".

11

0:30

Operator experience

"Aqib built the command center: live alerts sorted by risk, voice control, heatmaps, and an evidence locker."

Give Aqib credit by name.

12

0:20

Routing

"Each event goes to the right number: fire to 16, police events to 15, accidents to Edhi 115, crowds to 1122."

Keep it short.

13

0:30

Trust

"We are behavioural, not biometric — no face recognition. AI suggests, a human decides, and evidence cannot be secretly changed."

Slow down here; ethics matters to judges.

14

0:30

Roadmap

"We know our limits: fire numbers can improve, two reports are pending. Next: more training, local Karachi data, and a city pilot."

Saying limits first builds trust (اعتماد).

15

0:15

Close

"Cameras are already there. VisionGuard gives them a brain. Built by Moiz, Areeba, Aqib and me. Thank you — questions?"

Stop talking. Smile. Wait.

TOTAL TIME

About 7–8 minutes. For 3 minutes: slides 1, 2, 3, 5, 6, 9, 15. For 1 minute: 1, 3, 15 + one demo clip.

Q&A drill (سوال و جواب کی مشق)

Use the **L.A.B.** method: **Listen** (سنیں) → **Accept** (مانیں) “Good question” → **Bring back** (واپس لائیں) to a strength. Under 30 seconds.

#	Judge question	Your 30-second answer (key points)
1	What did YOU build?	Pipeline, async worker, cameras, 12-event engine, risk, audio, voice, evidence, server; connected everyone's work.
2	Why YOLOv8, not RT-DETR?	Fast on a 4 GB GPU; one tool for detection, pose, classification; transformers need more compute.
3	Your fire mAP is only 16% — why?	Hard dataset with tiny fires; every box counts; v2 is 2× v1; alerts decided over time; plan: more epochs, local data.
4	What is precision vs recall?	Precision: when it says fire, how often right. Recall: of all fires, how many found. We prefer fewer false alarms.
5	Hugging or sports = fight?	Body-size distance + fast wrists + repeats 1.5 s + violence classifier ≥ 0.70 .
6	Weapon accuracy?	Model works in demo; formal report not finished; I won't give an unproven number.
7	Is this mass surveillance?	Behavioural, not biometric; no face database; human confirms.
8	1000 cameras?	Each camera independent; add edge GPUs; server receives only alerts.
9	Internet goes down?	Detection is local; only Gemini report/voice uses internet, with offline fallback.
10	How is evidence tamper-proof?	SHA-256 fingerprint; one pixel change = different hash.
11	How did you get more speed?	Display thread + background specialist thread; 7 → 10 FPS, 48% fewer dropped frames.
12	What is new here?	Many threats + two-stage check + time rules + evidence + routing, for local cities, on a small GPU.

Rehearsal plan & score sheet

Day	Practice (مشق)	Goal
Day 1	Read the whole book once. Read the script out loud 3 times.	Know the story
Day 2	Practice with slides and a timer. Record yourself on phone.	Finish on time
Day 3	Practice the live demo 5 times. Break it on purpose once.	Calm demo
Day 4	Q&A drill: a friend asks the 12 questions + 5 random ones.	Answers < 30 s
Day 5	Full run in front of 2–3 people. Fill the score sheet.	Score 4+ everywhere
Final day	One light run only. Sleep well.	Fresh mind

Score sheet

Skill (1 = weak, 5 = strong)	Run 1	Run 2	Run 3	Run 4
Hook is strong and slow (تیز نہیں)	—	—	—	—
Problem is clear in 40 seconds	—	—	—	—
Demo: says where to look	—	—	—	—
Explains “how it works” simply	—	—	—	—
Says real numbers correctly (no fake numbers)	—	—	—	—
Answers in under 30 seconds	—	—	—	—
Eye contact (آنکھوں سے رابطہ) with judges	—	—	—	—
Finished on time	—	—	—	—
Calm when something breaks	—	—	—	—

Day-of checklist

- ☐ Laptop charged, notifications off
- ☐ Backend running (python prototype/run_web.py)
- ☐ Website running (npm run dev), logged in
- ☐ Desktop app warmed up with fire.mp4

- ☐ Backup demo video on laptop + USB + phone
- ☐ Slides PDF open in full screen
- ☐ Water, phone silent

Full knowledge guide

Simple English is used. Hard words have Urdu meaning in brackets, like this: *detect* (پتا لگانا).

Introduction (تعارف)

What is this document?

This is a guide for **Abdul Rafay** to present (پیش کرنا) VisionGuard ULTRA alone, for the full team. You can use it with judges at the stall, on stage, in a viva, or on an online call. Read it fully one time. Then practice (مشق) sections 2 and 4 out loud. Keep the small card (section 7) in your pocket.

Our project in 30 seconds

VisionGuard ULTRA is an AI system. It makes normal CCTV cameras smart. - It can **detect** (پتا لگانا) fire, fights, guns, big crowds, road accidents, and bags left alone. - It gives a **risk score** (خطرے کا نمبر) from 0 to 100. - It saves the video as **evidence** (ثبوت) that nobody can change secretly. - It sends the alert to the right number: **Police 15, Fire 16, Rescue 1122**.

Made by Abdul Rafay, Moiz, Areeba and Aqib for the Alibaba AI Hackathon 2026.

How to introduce yourself (say this first)

"Assalam-o-Alaikum. My name is **Abdul Rafay**. I am the **team lead** of team **VisionGuard**. I built the main AI system — the part that runs all the models together in real time. My team is **Moiz**, who trained our AI models, **Areeba**, who prepared and tested the data, and **Aqib**, who made the website. Today I will show you how we make normal CCTV cameras smart. Our system can **find a problem, decide, and send help** — very fast."

Tips: Smile. Say your name slowly. Keep it short (15–20 seconds).

Your work vs. team work — know the difference (فرق)

Judges may ask "What did **you** do?" Be clear and fair (منصفانہ):

Part	Who made it	How you say it
AI pipeline (<code>pipeline.py</code>), camera input, Event Engine (12 checks), voice commands, FastAPI server	You (Abdul Rafay)	"I built..."
Training fire, gun and fight models	Moiz	" Moiz trained... I connected them to the system."

Part	Who made it	How you say it
Data cleaning, augmentation, testing, false-alarm study	Areeba	"Areeba prepared and tested..."
React dashboard, design, control room screen	Aqib	"Aqib designed..."

Always give credit (کریڈٹ) to teammates by name. Judges like a lead who respects the team.

If a judge asks something from a teammate's part

You know the basic answer (section 3). If it goes very deep, say:

"Moiz did the training in detail. The main idea is... If you want, I can share his exact training settings later."

How to use this guide

If you have...	Read this
5 minutes	Section 7 (small card) + Section 4 (if demo breaks)
30 minutes	Sections 1, 2, 5, 7
One full day	Everything. Practice questions (section 3) with a friend.
Before any viva / deep technical judge	FULL PROJECT EXPLAINED (A to J) — read it twice

FULL PROJECT EXPLAINED — Start to End (پورا پروجیکٹ شروع سے آخر تک)

Abdul Rafay, this part is your **knowledge base** (معلومات کا خزانہ). You will not say all of it in a 3-minute talk. But if a judge asks about **any** part — your part or Moiz's, Areeba's or Aqib's — the answer is here.

A. The whole journey of one video frame (ایک تصویر کا پورا سفر)

Think of the system like a factory line (کارخانے کی لائن). One camera frame goes in, and an alert comes out.

1. CAMERA → frame comes in (webcam / RTSP CCTV / video file)
2. YOLOv8s → finds people, cars, bikes, bags (boxes + confidence)
3. ByteTrack → gives each object a fixed ID and follows it
4. SPECIALISTS → Pose (17 body points), Fire/Smoke, Weapon, Violence, Normal-scene
5. AUDIO → YAMNet listens for gunshot, scream, explosion, glass break
6. EVENT ENGINE → 12 detectors check rules over TIME (not one frame)
7. RISK SCORE → 0-100 → LOW / MEDIUM / HIGH / CRITICAL
8. ACTION → route to department, save evidence (SHA-256), show on dashboard, Gemini writes a short report, operator confirms

Now each step in simple words:

Step 1 — Camera input (کیمرے سے تصویب) · built by Abdul Rafay - Code: `prototype/camera/` (`webcam.py` , `rtsp.py` , `video_file.py`). - Each camera runs in its own **thread** (الگ کام کی لائن). If an RTSP camera disconnects, it tries to **reconnect** (دوبارہ جڑنا) automatically. - Old frames are dropped, so the system always works on the **newest** frame (no delay building up).

Step 2 — Object detection (چیزیں ڈھونڈنا) with YOLOv8s - Code: `prototype/ai/detector.py` . Model: `yolov8s.pt` (pre-trained on COCO dataset, 80 types of objects). - It draws a **bounding box** (ڈب) around each person, car, bike, bag, and gives a **confidence** (یقین) from 0 to 1. - We keep only boxes with confidence ≥ 0.35 (`DETECTION_CONF_THRESHOLD`). Image size **480** for speed. - **Why YOLO?** "You Only Look Once" — it looks at the whole image one time, so it is very fast.

Step 3 — Tracking (پیچھا کرنا) with ByteTrack - Code: `prototype/ai/tracker.py` . Detection and tracking happen in **one call** (`model.track()`), which doubled the speed. - ByteTrack gives every person a fixed **ID** (e.g. Person #7). It also uses low-confidence boxes to not lose a person in a blur. A lost person is remembered for 30 frames (~1 second). - **Why we need IDs:** to know *how long* someone stands (loitering), *how fast* they move (robbery run), and *who left a bag*.

Step 4 — Specialist models (ماہر ماڈلن) · trained by Moiz, connected by Abdul Rafay - Code: `prototype/ai/specialist_worker.py` . They run in a **separate background thread**, one after another on the GPU. - The worker is **smart** (ہوشیار): it picks what to run based on the scene. Pose runs only when people are present. Fire runs every AI frame (safety first). Weapon runs every 3rd AI frame. - Every result has an **age** (عمر). A result older than 0.75 s is not drawn, older than 1.2 s is not used for events. So old results never cause wrong alerts.

Specialist	What it does	Model
Pose	17 body points (shoulders, elbows, wrists, hips...)	<code>yolov8s-pose.pt</code> (pre-trained)
Fire/Smoke	Boxes around fire and smoke	<code>fire_smoke_best.pt</code> — trained by Moiz
Weapon	Knife, pistol, rifle	<code>weapon_detection_best.pt</code> — trained by Moiz
Violence	Looks at a person crop (کٹی ہوئی تصویر) → violent / not violent	<code>violence_classifier_best.pt</code> — trained by Moiz
Normal scene	Checks if the scene is normal (to stop false alarms)	<code>normal_scene_verifier_best.pt</code>

Step 5 — Audio (آواز) · built by Abdul Rafay - Code: `prototype/ai/audio.py` . Google's **YAMNet** model listens to the microphone in 1-second pieces (16 kHz). - Emergency sounds: gunshot, explosion, scream, crash, glass breaking, siren. Confidence must be ≥ 0.4 . - Sounds from the last 5 seconds are **fused** (ملاپا جاتا ہے) with video. Example: gun seen + gunshot heard = higher risk.

Step 6 — Event Engine (واقعات کا انجن) · built by Abdul Rafay - Code: `prototype/events/engine.py` + one file per event in `prototype/events/` . **12 detectors:** fire, car accident, bike accident, fight, robbery, kidnapping, weapon threat, crowd crush, loitering, vehicle obstruction, abandoned object, audio emergency. -

How every detector works (same pattern, `events/base.py`): 1. Take **signals** (اشارے) from the current frame. 2. Put them in a **time window** (وقت کی کھڑکی) — e.g. last 5–8 seconds. 3. Event must **continue** for at least **1.5 seconds** (`EVENT_MIN_PERSISTENCE_SECONDS`). 4. A short **grace period** (رعایتی وقت) — if YOLO misses one or two frames, the timer does not reset. 5. Calculate **risk score 0–100**. Below 25 = ignored. - To save time, a detector is skipped if it cannot happen (e.g. fight needs at least 2 people).

How the main detectors decide — simple rules:

Event	Rule in simple words
Fight	2+ people are close (distance measured by body size , so it works near and far) + wrists move fast (> 16 px/frame) or arms raised + it repeats for 1.5 s. Violence classifier (≥ 0.70) makes it stronger.
Fire	Fire/smoke confidence ≥ 0.35 , area between 0.2% and 65% of the screen (very big = lighting glitch), and checks if the fire is growing (بڑھ رہی) ($\times 1.3$).
Weapon threat	Weapon box within 250 px of a person, for 0.5 s (very fast because it is very dangerous).
Crowd crush	Number of people vs. zone capacity (50). 70% = warning, 95% = danger.
Car accident	2 vehicles overlap ($\text{IoU} \geq 0.15$) + sudden speed drop (> 15 px/frame).
Bike accident	Rider leans more than 45° + speed drop.
Robbery	Vehicle stops 3+ s, 2+ people run fast (> 30 px/frame) toward someone.
Kidnapping	Fast approach + struggle (body points shaking a lot) + forced movement.
Loitering	Person stays in a small area (< 50 px movement) for 5 minutes.
Abandoned object	Bag alone for 60 s, owner more than 200 px away.
Vehicle obstruction	Vehicle stopped (speed < 2) for 10+ s.
Audio emergency	YAMNet emergency sound with confidence ≥ 0.4 .

Step 7 — Risk score (خطرے کا نمبر) | Score | Level | What happens | |---|---|---| | 85–100 | **CRITICAL** | Auto-dispatch suggestion (`RISK_AUTO_DISPATCH_THRESHOLD = 85`) | | 50–84 | **HIGH** | Team review | | 25–49 | **MEDIUM** | Shown as warning | | 0–24 | **LOW** | Ignored |

Step 8 — Action (کارروائی) - **Department routing** (`config.py` → `DEPARTMENT_ROUTING`):

Event	Department	Dial
Fire	Fire Brigade	16
Fight, robbery, weapon, kidnapping, audio	Sindh Police / Rangers	15

Event	Department	Dial
Car / bike accident	Edhi / Rescue	115
Crowd crush	Disaster Management	1122
Vehicle obstruction	Traffic Police	1915
Abandoned object	Police / Bomb Squad	15

- **Evidence** (ثبوت) (`prototype/output/evidence.py`): the system always keeps the last **5 seconds** of video in memory. When an event happens, it saves **5 s before + 3 s after** as a clip, plus a picture. Then it makes a **SHA-256 hash** — a unique digital fingerprint. If even one pixel changes, the hash changes. So nobody can secretly edit the evidence.
- **Database** (`prototype/events/incident_db.py`): every incident is saved in SQLite. It is also used to make **heatmaps** (گرم نقشے) — which areas have most incidents.
- **Gemini report** (`prototype/voice/narrator.py`): turns the event into a short sentence, like "Fight in Sector 4, Police 15 informed."
- **Human confirms**: the operator can **confirm** or **dismiss** (رد کرنا) the alert. AI suggests, human decides.

B. Speed — the async pipeline (الگ الگ چلنے والا نظام — رفتار) • Abdul Rafay

Problem: First version ran all models one after another on the main loop. Video became slow and jumpy.

Solution: We **decoupled** (الگ کیا) it: - **Thread 1 (display)**: reads camera, runs fast YOLOv8s + ByteTrack, shows video smoothly. - **Thread 2 (specialist worker)**: runs heavy models (pose, fire, weapon, violence) in the background on the newest frame. - AI runs on every 2nd frame (`AI_PROCESS_EVERY_N_FRAMES = 2`). GPU uses **FP16 (half precision / آدھی درستگی)** for speed.

Our real benchmark (`prototype/baseline.json` vs `prototype/async.json` , Quadro T1000, 720p fire video, 300 frames):

	Old (synchronous)	New (async)	Change
Display FPS	7.0	10.0	+43% faster
Dropped frames	866	450	48% fewer
Total time	42.7 s	29.9 s	30% less
GPU memory	144 MB	157 MB	very small

After this, we lowered image size to 480 and added smart scheduling. Our target is 22+ FPS on the T1000 (`config.py`). **Say clearly:** "Our recorded benchmark shows 7 → 10 FPS for the first async version; after more tuning we target 22+ FPS on this small GPU." Do not say 30 FPS unless you show it live.

C. Voice control and backend (آواز سے کنٹرول اور سروں) • Abdul Rafay

- **Voice:** browser listens (Web Speech API, free) → text goes to **Gemini** (`voice/interpreter.py`) → Gemini returns a **JSON command** like `show_cameras` or `status_summary` → `voice/executor.py` does it. If internet is gone, a simple **offline word-matching** (regex) backup understands basic commands.
- **Backend:** `prototype/web/server.py` (FastAPI). Main parts:
 - `/video_feed/{camera_id}` — live video to the browser
 - `/ws/alerts` — **WebSocket:** alerts are **pushed** to the screen instantly (no refresh)
 - `/api/cameras/connect` — add a new camera
 - `/api/evidence/chain/{event_id}` — check evidence hash
 - `/api/heatmaps/karachi` — incident heatmap
 - `/api/login` , `/api/users` — login and users

D. How Moiz made the models (موئز نے ماڈلز کیسے بنائے)

Method — transfer learning (پہلے سے سیکھے ماڈل کو آگے سکھانا): Moiz did not start from zero. He took YOLOv8s, which already knows general shapes from millions of images, and **fine-tuned** (مزید تربیت) it on our special data. This needs less data and less time.

Where: Google Colab, free **T4 GPU**.

Model	Type	Data	Input	Training
Fire/Smoke v2	YOLOv8s detection	D-Fire (~16,000 clean images) + Roboflow fire & smoke	640×640	80 epochs
Weapon	YOLOv8s detection	Weapon CCTV + knife + pistol sets (~7,100 clean images)	640×640	60 epochs
Violence	YOLOv8s-clc classification	RWF-2000 + Real Life Violence + CCTV violence (~12,000 images)	224×224	50 epochs

- **Epoch** (ایک مکمل چکر) = the model sees the whole dataset one time.
- **Detection vs classification:** detection says *where* (box) + *what*. Classification only says *what* for the whole picture (violent / not violent).
- **Why fire v2?** Version 1 used **colour only (HSV)** — red/orange = fire. It gave false alarms on red clothes, sunsets and car lights, and could not see smoke. v2 is a neural network that learned the **shape and texture** of fire and smoke.
- **Two-stage idea for fights:** pose (cheap, fast) first finds suspicious arm movement → only then the violence classifier checks the **person crop**. Max 3 crops per frame to save GPU.

E. How Areeba prepared and tested the data (اریبہ نے ڈیٹا کیسے تیار اور ٹیسٹ کیا)

1. Cleaning (صفائی) — `data_science/corrupt_file_scanner.py` - Checks for broken images, empty (0-byte) files, **duplicate** (ایک جیسی) images, and wrong label lines. - **Real results:** D-Fire: 16,027 images, 0 broken, 0 duplicates, 35 bad label lines removed. Weapon: 7,121 images, **5 duplicates removed** → 7,116 clean.

2. Augmentation (ڈیٹا بڑھانا) — `data_science/augmentation_pipeline.py` (Albumentations library) - Makes new copies of images with **rain, fog, shadow, brightness change, rotate, zoom**. Real CCTV has bad weather and bad light, so the model must learn these. - **Real result:** 4,304 weather images made.

3. Class balancing (برابری) — `data_science/class_balancer.py` - If one class has many more images (e.g. more pistols than knives), the model becomes biased. The script counts classes and **oversamples** (کم والی کلاس کو بڑھانا) the small ones.

4. False-positive analysis (غلط الارم کی جانچ) — `data_science/false_positive_analyzer.py` - Looked at 7 saved alerts. Found: crowd crush alarm fired twice in 2 s (duplicate), and 2 fight alarms in **dark frames** (low light). - **Changes applied in `config.py`** : - Crowd danger 0.90 → **0.95** - Event persistence 1.0 s → **1.5 s** - Detection confidence 0.30 → **0.35** - This is **data-driven tuning** (ڈیٹا کی بنیاد پر بہتری) — we changed limits because of real evidence, not guessing.

5. Evaluation (جانچ) — `data_science/model_evaluation.py` - Calculates mAP, precision, recall, confusion matrix and PR curves.

F. Accuracy words — explained simply (درستگی کے الفاظ)

Example: 100 real fires in test images.

Word	Simple meaning	Urdu
TP (True Positive)	Fire was there, model said fire ✓	صحیح پکڑا
FP (False Positive)	No fire, model said fire ✗ (false alarm)	غلط الارم
FN (False Negative)	Fire was there, model missed it ✗	چھوٹ گیا
TN (True Negative)	No fire, model said no fire ✓	صحیح چھوڑا
Precision	When model says "fire", how often is it right? = $TP \div (TP + FP)$	درستگی
Recall	From all real fires, how many did it find? = $TP \div (TP + FN)$	پکڑنے کی شرح
F1-score	One number that balances precision and recall	توازن کا نمبر
IoU	How much the model's box overlaps the real box (0 to 1)	ڈبوں کا ملاپ
mAP@50	Average precision when a box counts as correct if IoU ≥ 0.5	اوسط درستگی
mAP@50-95	Same, but strict — average over IoU 0.5 to 0.95	سخت اوسط درستگی
Accuracy (Top-1)	For classification: % of images with the correct class	درستگی فیصد
Confusion matrix	A table of TP, FP, FN, TN	الجھن کی جدول

Precision vs recall trade-off (سودا): if we raise the confidence limit, we get fewer false alarms (precision ↑) but miss more events (recall ↓). For a city, we prefer **fewer false alarms**, and we fix missed events with **time checks** and **many cameras**.

G. Our real results — say them honestly (ایمانداری سے — اصل نتائج)

Fire/Smoke v2 (from `data_science/ML_Evaluation_Report.md` , D-Fire validation):

Metric	Fire v2 (AI model)	Fire v1 (colour only)	Change
mAP@50	16.0%	~8%	2× better
mAP@50-95	7.6%	~3%	2.5× better
Precision	47.6%	~25%	+90%
Recall	25.5%	~35%	lower (by design: fewer false alarms)

Per class: fire precision **56.1%**, recall 22.9% (2,878 boxes) · smoke precision **39.1%**, recall 28.1% (2,309 boxes).

How to explain these numbers (very important):

"Our fire model doubled the old colour method on mAP and almost doubled precision. The image-level numbers look low because D-Fire is a hard dataset — many fires are tiny, far away or partly hidden, and every single small box is counted. But in our system we don't alert on one frame. The fire must stay for 1.5 seconds and the event engine checks its size and growth. So even if a frame is missed, the event is still caught over many frames. Our next step is more training epochs, a bigger model and local Karachi data to improve these numbers."

Weapon model and violence classifier: training is done / in progress by Moiz, but the **formal evaluation numbers are not finished yet**. Say:

"The weapon and violence models are working in the demo, but their final test report is still being completed. I don't want to give you a number I can't prove."

False alarm rate: target is under 3–5%. It is **still being measured** on a bigger test set. Do not say a fixed % yet.

Sign-off goal (`Accuracy_SignOff_Report.md`): mAP50 $\geq 70\%$ for detection, Top-1 $\geq 85\%$ for classification. We have **not reached it yet** — this is our honest roadmap.

⚠ Golden rule: **real numbers + honest explanation** wins more trust than big fake numbers.

H. How Aqib made the website (عاقب نے ویب سائٹ کیسے بنائی)

- **Tools:** React + Vite (fast website tools), code in `frontend/src/`.
- **Pages** (`frontend/src/pages/`):

Page	What it shows
Login.jsx	Secure login (only allowed users)
Dashboard.jsx	Camera grid (matrix), live alerts list, risk colors
LiveView.jsx	One camera big, with boxes and skeletons
Incidents.jsx	All past incidents, evidence clips, SHA-256 hash
Registry.jsx	List of all cameras and their locations
ConnectCamera.jsx	Add a new camera (RTSP link / webcam / test video)

- **Components:** CameraTile.jsx (one camera box), AlertList.jsx (alerts sorted by risk), LocationTree.jsx (city → sector → camera).
- **Design ideas:** dark control-room theme (easy for eyes at night), **red / orange / green** colours for risk, most dangerous alert at the top, layouts 1×1 / 2×2 / grid, voice box.
- **How it connects:** video comes from /video_feed/... ; alerts come live through **WebSocket** /ws/alerts — no page refresh needed.

I. Who did what — one-line answers (کس نے کیا کیا)

Member	One line to say
Abdul Rafay (me)	"I designed the full system and built the AI pipeline, async worker, camera input, 12-event engine, risk scoring, audio fusion, voice control, evidence system and FastAPI server, and I connected everyone's work together."
Moiz	"Moiz trained our custom models — fire/smoke, weapon and violence — using transfer learning on Colab T4 GPUs."
Areeba	"Areeba cleaned and checked the datasets, made weather augmentations, balanced classes, analysed false alarms, and ran the evaluation that gave us our metrics."
Aqib	"Aqib designed and built the React command-center website — dashboard, live view, incidents, camera registry and voice box."

J. Weak points — say them before the judge does (کمزوریاں)

Judges respect a team that knows its limits (حدود): 1. Fire model numbers are still low on the hard D-Fire test → plan: more epochs, bigger model, local data. 2. Weapon and violence final reports are not finished → plan: run model_evaluation.py for both. 3. Small weapons far from the camera are hard to see → plan: higher resolution crops near people. 4. Dark / night video gives more false fights → plan: low-light augmentation and IR cameras. 5. Not yet tested on a real city camera network → plan: pilot project. 6. Pixel rules (like 200 px distance) depend on camera position → plan: per-camera settings (calibration / ترتیب).

0. Before You Start — Checklist (30 minutes before)



Thing to check

- ☐ Laptop is fully charged. Charger is plugged in. Notifications are off.
- ☐ Start backend (سروں): `python prototype/run_web.py` . Open `http://localhost:8000/docs` to check.
- ☐ Start website: `cd frontend && npm run dev` . Open `http://localhost:5173` and log in.
- ☐ Open the desktop app one time: `python prototype/visionguard_app.py` and play `fire.mp4` (so it is ready and fast).
- ☐ **Backup video (متبادل ویڈیو)** of the full demo is saved on laptop, USB and phone.
- ☐ Demo videos are in `prototype/Videos/` (fire, fighting, gun, crowded, thugs).
- ☐ Browser tabs are open in this order: Dashboard → Live View → Incidents → Registry. Close other tabs.
- ☐ Screen zoom is 110–125%, so judges can read.
- ☐ Water bottle with you. Phone on silent.

Remember: The demo only helps. Your **story** is the main thing. If the demo stops, your story can still win.

1. The Plan (remember this order)

HOOK (20s) → PROBLEM (40s) → SOLUTION (40s) → LIVE DEMO (2–3 min)
→ HOW IT WORKS (1 min) → BENEFIT & FUTURE (30s) → END (15s) → QUESTIONS

Hook (ہک) = the first line that catches attention (توجہ کھینچنا).

Change length by your time:

Time you get	What to say
1 minute	Hook + 1 line problem + solution + 1 video (fire or gun) + end
3 minutes	Full plan below, show 2 videos
5–7 minutes	Full plan + 3–4 videos + system diagram + future plans
10+ minutes (viva)	Everything + show your own code (<code>prototype/ai/pipeline.py</code> , <code>prototype/events/engine.py</code> , <code>prototype/voice/interpreter.py</code> , <code>prototype/web/server.py</code>) + team's data part

2. What to Say — Step by Step

Step 1 — Hook (about 20 seconds)

"Karachi has more than 20 million people and thousands of CCTV cameras. But these cameras only **record**. Nobody is really **watching**. A person watching screens gets tired after about 20 minutes. So the camera sees the crime... but nobody acts."

Tip: Stop for 2 seconds after "nobody acts". Look at the judges.

Step 2 — Problem (مسئلہ) (about 40 seconds)

- Normal CCTV only shows the crime **after** it happens. It does not stop it.
- People who watch screens get tired. One person cannot watch 50 screens.
- Help comes late, because first someone must see it, then call 15 / 16 / 1122.
- There are many dangers: fights, robbery, fire, big crowds, accidents, bags left alone.

Step 3 — Solution (حل) (about 40 seconds)

"**VisionGuard ULTRA** makes cameras smart. It watches all cameras at the same time. It finds fire, fights, guns, crowds, accidents and left bags in **real time** (فوری طور پر). It gives a risk score, saves the proof, and sends the alert to the right place — Police 15, Fire 16, Rescue 1122."

Repeat these three words: **Detect** → **Decide** → **Dispatch** (پتا لگائی → فیصلہ کرو → مدد بھیجو).

Step 4 — Live Demo (2–3 minutes)

#	What you click	What you say
1	Open Dashboard (camera grid)	"This is the control room screen. AI is checking every camera right now."
2	Play fire.mp4 , wait for red box	"It found fire and smoke. It checks for a few seconds first, so it does not give a false alarm (غلط الارم). Alert goes to Fire Brigade 16 ."
3	Play gun.mp4 or fighting.mp4	"Here is a gun / a fight. For fights, we see 17 points on the body (skeleton / ڈھانچہ). If the arms move very fast, a second model checks it again. Two checks = fewer mistakes."
4	Point at risk score and alert list	"Every event gets a score from 0 to 100. The most dangerous one is always on top."
5	Open Incidents page	"Every event saves a video clip as evidence (ثبوت). We add a SHA-256 hash — a digital fingerprint (ڈیجیٹل انگوٹھے کا نشان). If someone changes the video, the fingerprint will not match."
6	(Optional) Say " What is going on? "	"The operator can talk to the system. No need to click."
7	(Optional) crowded.mp4	"It checks crowd size and movement, to warn before a stampede (بھگدڑ)."

Demo rules: - Tell the judge where to look: "Look at the top left. The box is red now." - Never say "it usually works". If it is slow, say: "While it loads, let me explain what it is doing..." - Only 2–3 videos in a 3-minute talk. Show less, explain better.

Step 5 — How It Works (about 1 minute)

Show or draw this:

```
CAMERAS / MIC / SOS
↓
YOLOv8 (finds objects) → ByteTrack (follows each person) → Pose (17 body points)
↓
Special models (fire/smoke, gun, fight)
↓
Event Engine (12 checks, over time) → Risk score 0–100
↓
Send alert · Save evidence · Show on dashboard · AI short report
```

Main points: 1. **Two parts run separately (الگ الگ)** — video shows on one side, AI works on the other side. So the video never freezes (رکتی نہیں). 2. **Two checks** — first a quick check, then a strong check. This means fewer false alarms. 3. **Time check** — the event must continue for some time, not only one frame (تصویر). 4. **Works on a small computer** — tested on an NVIDIA Quadro T1000 graphics card (4 GB). Async design made it 43% faster.

Step 6 — Benefit & Future (فائدہ اور مستقبل) (about 30 seconds)

- **Benefit:** help comes faster, workers get less tired, cities can use the cameras they already have. No new cameras needed.
- **Future:** test it on real city cameras, use it in hospitals, schools and factories (see `prototype/MULTI_SECTOR_EXPANSION_PLAN.md`), connect directly to the helplines, collect more local Karachi data.

Step 7 — End (about 15 seconds)

"Cameras are already there. VisionGuard gives them a brain. **Find, decide, send help — in less than one second.** This was built by Moiz, Areeba, Aqib and me. Thank you. I am happy to take your questions."

Then **stop talking**. Silence (خاموشی) invites questions.

3. Questions and Answers (سوال و جواب)

3.1 The easy method for every question — L.A.B.

1. **Listen (سنیں)** — listen to the full question. Do not interrupt (بیچ میں نہ بولیں).
2. **Accept (مانیں)** — say "Good question" or "That is an important point."
3. **Bring back (واپس لائیں)** — answer in 1–3 short lines, then go back to a strong point of the project.

Keep each answer **under 30 seconds**.

3.2 Common questions and simple answers

Technical questions (تکنیکی سوالات)

Question	Simple answer
Why YOLOv8 and not a Transformer (RT-DETR) or Faster R-CNN?	YOLOv8 is fast and needs less memory. Faster R-CNN is too slow for many live cameras. Transformers need a stronger computer. YOLOv8 can do objects, body points and classification in one tool.
What about false alarms? Hugging, playing, sports?	We use three steps: (1) fast arm movement starts a check, (2) a second model checks the person again, (3) the event must continue for some time. We can also change the limits in <code>config.py</code> .
How fast is it?	"Our benchmark on a small Quadro T1000 GPU: the async design made it 43% faster (7 → 10 FPS at 720p) and dropped half the lagging frames. With 480p and smart scheduling we target 22+ FPS." See section B.
Which data did you use?	Fight: RWF-2000 and other violence video sets. Fire/smoke: D-Fire. Guns: public weapon image sets. We trained on Google Colab (T4 GPU). We removed broken files and added blur and rain effects to look like real CCTV.
How accurate (درست) is it?	"Our fire model: mAP@50 16%, precision 47.6% — two times better than the old colour method. D-Fire is a hard dataset with tiny fires; our time checks catch the event over many frames. Weapon and violence reports are still being finished." See section G.
How does tracking (پیچھا کرنا) work?	ByteTrack gives each person or car a fixed ID number. So we can see how long someone stays, if a bag is left, and how fast things move.
How do you find a left bag?	We follow the bag by its ID. If it does not move and its owner is gone for some time, we give an alert.
Can it work with 1000 cameras?	Yes. Each camera works on its own. We add more small GPU computers. The main server only gets alerts and small pictures, not full video.
What does Gemini (the LLM) do?	It writes a short report in normal language, and it understands voice commands. Finding the danger does not need Gemini.
What if internet stops?	Detection works on the local computer. Only the voice and report part uses internet, and it has an offline backup.

Privacy and ethics questions (رازداری اور اخلاقیات)

Question	Simple answer
Is this spying (جاسوسی) on everyone?	No. We check actions , not who the person is. We do not use face recognition. We do not keep a face database.

Question	Simple answer
Is it biased (جانبدار)?	We look at actions and objects, not skin, face or clothes. We still want more local data and more testing in different light and crowds.
Who can see the data?	Only people with login. Evidence has a hash, so any change can be caught. The city decides how long data is kept.
What if it sends police by mistake?	The system only suggests . A human operator checks and confirms. AI helps, humans decide.

Business questions (کاروباری سوالات)

Question	Simple answer
Who will buy it?	Safe City projects, police control rooms, malls, universities, hospitals, factories.
What is the cost?	It uses the cameras that already exist. Cost is only small GPU computers and support. Cheaper than many people watching screens 24 hours.
What is different from others?	Many dangers in one system, two checks, safe evidence, voice control, made for local needs (15, 16, 1122), works on a small computer.
Next 6 months?	Test with real city cameras, collect local data, finish the accuracy report, connect to helplines.

3.3 Difficult moments (مشکل حالات)

Situation	What to do / say
Judge asks "What did YOU do?"	"I am the team lead. I built the main AI pipeline, the event engine with 12 checks, the voice commands and the server. I also connected everyone's work into one system."
You do not know the answer	"Good question. I don't want to guess. What I know is... I will send you the exact details later." Never make up numbers.
Judge says "this is not new"	"Yes, finding objects is not new. Our new part is putting many dangers, two checks, evidence and alerts together in one fast system for our cities."
Judge doubts (شک) a number	"Fair point. These numbers are from our test computer and demo videos. A real city test is our next step." Being honest is better than fighting.
Judge is angry or rude	Stay calm. Speak slowly. Say thank you. Answer the main point. Do not argue (بحث نہ کریں).
Two questions together	"Let me answer one by one. First..."
Question not about our project	Connect it to the project in one line, or say "This is not in our project now, but it can be added like this..."

Situation	What to do / say
You forget what to say	Remember the order: Problem → Solution → Demo → How → Benefit. Say "Let me show you the main part" and start the demo.
Time is almost over	Go to the End (Step 7) now. Do not rush.

4. If the Demo Breaks (Plan A → B → C)

Problem	Quick fix (20 seconds)	If it still fails
Backend stopped	<code>python prototype/run_web.py</code>	Plan B
Website stopped	<code>cd frontend && npm run dev -- --host</code>	Use desktop app: <code>python prototype/visionguard_app.py</code>
Video is slow	Use 1×1 layout, close other apps	Use simple player: <code>python prototype/run.py --source video --file prototype/Videos/fire.mp4</code>
Mic not working	Type the command in the voice box	Skip voice. It is optional (اختیاری).
Laptop crash / no GPU	—	Plan C: play the backup video and explain it
Projector problem	Turn your laptop to the judges	Draw the system on paper or board

Say this when you switch: "Let me show you the recorded run, so we don't waste your time. This is the same system."

5. Numbers to Remember (say them carefully)

- **12** event detectors: fire, fight, weapon, robbery, kidnapping, crowd crush, car accident, bike accident, loitering (بلا وج گھومنا), abandoned object, vehicle obstruction, audio emergency.
- **3** models trained by Moiz: fire/smoke, weapon, violence (+ pre-trained YOLOv8s, YOLOv8s-pose, YAMNet).
- **17** body points per person.
- **1.5 s** — an event must continue this long before alert. **0.5 s** for weapons.
- **Risk:** 85+ critical, 50–84 high, 25–49 medium.
- **Evidence:** 5 s before + 3 s after, SHA-256 fingerprint.
- **Speed (real benchmark):** async pipeline **7** → **10 FPS** (+43%), dropped frames **866** → **450** on Quadro T1000 at 720p. Target 22+ FPS after tuning.
- **Fire v2:** mAP@50 **16.0%** (2× old), precision **47.6%** (old ~25%), recall **25.5%**.
- **Data:** D-Fire 16,027 clean images · Weapon 7,116 (5 duplicates removed) · 4,304 weather images made.
- **Tuning:** crowd 0.90 → 0.95, persistence 1.0 → 1.5 s, confidence 0.30 → 0.35.

- **Helplines:** Police **15**, Fire **16**, Edhi **115**, Rescue **1122**, Traffic **1915**.

⚠️ Weapon and violence accuracy, and the false-alarm rate, are **not final yet**. Do not say a number for them. A wrong number makes judges stop trusting (اعتماد) you.

6. Body Language (جسمانی انداز) and Speaking

- Stand next to the screen, not in front of it. Keep your hands visible.
- Speak a little slower than normal. Stop for a moment after important lines.
- Look at the judges, not the screen. Look at the screen only to point.
- Smile at the start and at the end.
- Use short sentences. One idea in one sentence.
- If your English stops, it is okay. Take a breath and say it in simple words. Judges care about the idea, not perfect English.

7. Small Card (print this)

```
INTRO    Abdul Rafay, team lead, built AI pipeline. Team: Moiz, Areeba, Aqib.
HOOK     20 million people. Cameras record. Nobody watches.
PROBLEM  Cameras only record, people get tired, help comes late, many dangers.
SOLUTION Smart cameras: Find → Decide → Send help.
DEMO     Dashboard → fire.mp4 → gun/fight → risk score → Incidents (SHA-256) → voice
HOW      YOLOv8 → ByteTrack → Pose → Special models → Event Engine → Risk → Alert
          separate parts · two checks · time check · small computer
BENEFIT  Use old cameras, faster help, more areas in future.
END      "Cameras are there. We give them a brain." + thank team → questions.
BROKEN   run_web.py · npm run dev · desktop app · backup video
```